



FOR JAVA DEVELOPERS MOVING TO NESTJS

Fresh DB. Still 2.

Spring Boot to NestJS, episode 19: Testcontainers

A container gives you a clean database per run, not per test. Where you own it decides what shares it.

Two tests against a Postgres container, run twice: 1 then 2, and again 1 then 2, on both stacks. Runs are clean; tests inside a run still share.

One container per class or file isolates them, at the price of a start each. One shared container starts once and shares its data: counts followed the run order.

Container for the run, reset for the test.

In scope

- @Container and @ServiceConnection against PostgreSQLContainer
- Container lifecycle: per class, per file, per run
- Vitest globalSetup with provide and inject

Not in scope

- Container reuse between runs
- Other databases

Timings are this machine with a cached image; they vary.

01 The measurements

CONTAINER PER RUN, RUN TWICE (BOTH STACKS)	1, 2 and 1, 2
CONTAINER START, THIS MACHINE	about 1.1 to 1.6 s
SPRING, THREE CLASSES, OWN @Container	three starts; each class saw 1
NEST, THREE CONTAINER STARTS AND STOPS	about 4.1 to 4.5 s
SPRING, ONE CONTAINER IN A BASE CLASS	one start; 1, 2, 3 in class order
NEST, ONE CONTAINER FROM globalSetup	one start; 1, 2, 3 in file order, order changed between runs

Each test inserts one row, then counts.

02 The mapping

Spring declares a static container and lets @ServiceConnection wire the DataSource. Nest starts the container and passes its URI to TypeOrmModule.

● ● ● Ep18ContainerTest.java

```
@Container
@ServiceConnection
static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:18-alpine");
```

● ● ● fresh.spec.ts

```
container = await new PostgreSQLContainer('postgres:18-alpine').start();
TypeOrmModule.forRoot({ type: 'postgres', url: container.getConnectionUri(), entities: [Note],
synchronize: true }),
```

container per run, two tests, run twice: 1, 2 and 1, 2 on both stacks

Run on a real machine, 2026-09-30, by scripts/verify-testcontainers.sh in the course repository.

03 One container for the run

Started once in globalSetup, handed to every file. Pair it with a reset before each test so file order cannot leak into results.

● ● ● global-setup.ts

```
export async function setup(project: TestProject) {
  container = await new PostgreSQLContainer('postgres:18-alpine').start();
  project.provide('pgUri', container.getConnectionUri());
}
```

three files, one shared container: 1, 2, 3 in whichever order the files ran

Run on a real machine, 2026-09-30, by scripts/verify-testcontainers.sh in the course repository.

Before you trust a Testcontainers suite

1. A container per run is not isolation per test
2. Decide who owns the container: class, file, or run
3. For one container per run, use globalSetup with provide and inject
4. Reset data before each test
5. Expect results to depend on order if you share without a reset
6. Measure start-up on your machine, not a blog's

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-testcontainers.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	Testcontainers for Node.js https://node.testcontainers.org

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Docker is a trademark of Docker, Inc. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.